TASK 7 – Control Type Conversion and Precision 4 Marks

Bloom's Level: K4 – Analyse

Identify every place in the program where values of different types are combined.

Discuss:

• Implicit conversion (integer promotion and the usual arithmetic conversions)

• Integer division (for example units / 100) and where it gives a wrong result

• Explicit casting, and the effect of casting a large long to int

• Rounding rules when converting a fractional charge to paise

Show the output of at least three expressions that give an unexpected result, and explain each

one.

Alternative Concept and Complexity Analysis

If all calculations were done in float and rounded only once at the end, instead of in integer paise,

what would the solution be? Measure and analyse how the error in the total grows with the

number of bills n, and compare the time and space per value of each approach.

Evidence Expected

Identified conversion points, three demonstrated surprises with explanations, and a consistent rounding rule.

1. Type Conversion in C

Type conversion occurs when values of different data types are combined in an expression. In a billing program, conversion can occur when an int value is multiplied by a long rate, when integer values are divided, and when a large long value is assigned to an int.

Implicit Conversion

Implicit conversion happens automatically. For example:

int units = 150;

long rate = 225;

long charge = units * rate;

The int value is converted to long during multiplication. Integer types smaller than int may also undergo integer promotion before arithmetic operations.

Integer Division

Integer division discards the fractional part when both operands are integers.

printf("%d\n", 5 / 2);

Output: 2

The correct way to obtain a decimal result is:

printf("%.2f\n", 5.0 / 2);

Output: 2.50

Explicit Casting

Explicit casting converts a value to a specified type.

long amount = 500000;

int value = (int)amount;

If the long value is outside the range of int, converting it to int is implementation-defined in C. Therefore, a large amount should not be converted to int without checking its range.

Rounding to Paise

One rupee equals 100 paise. If a calculated charge contains a fraction of a paise, a consistent rounding rule should be applied. For example, round to the nearest paise, with exact half-paise cases rounded upward for non-negative charges.

2. Three Unexpected Expressions

Example program:

```
#include <stdio.h>


int main(void)
{
   int a = 5;
   long amount = 500000;
```

```
    double charge = 12.345;

    printf("1. Integer division: %d\n", 5 / 2);
    printf("2. Cast after division: %d\n", (int)(5.0 / 2));
    printf("3. Fractional charge: %.2f\n", charge);

    printf("4. Cast large long to int: %d\n", (int)amount);

    return 0;
}
```

Expected output on a typical system with a 32-bit int:

1. Integer division: 2

2. Cast after division: 2

3. Fractional charge: 12.35

4. Cast large long to int: 500000

Explanation:

1. 5 / 2 produces 2, not 2.5, because integer division discards the fractional part.
2. (int)(5.0 / 2) first calculates 2.5 as a floating-point value and then truncates it to 2.
3. Printing 12.345 with %.2f displays two decimal places. The displayed result depends on floating-point representation and the formatting rounding rules.
4. Casting 500000L to int is safe on a typical system with a 32-bit int, but a value outside the int range cannot be converted portably in this way.

3. Alternative: Use float for All Calculations

An alternative is to store charges in float, perform all calculations in floating point, and round only the final total to two decimal places.

However, binary floating-point cannot represent every decimal fraction exactly. Small errors may accumulate across many calculations.

If the error per bill is at most (\epsilon), the worst-case accumulated error across (n) bills can be bounded by approximately (n\epsilon), assuming the same error bound applies to each bill. Actual errors may be smaller or may cancel out.

4. Time and Space Complexity

Method	Storage per value	Calculation time	Precision
Integer paise using long	Usually 4 or 8 bytes, depending on the system	Constant time per arithmetic operation	Exact for whole-paise values, provided no overflow occurs
float calculations	Usually 4 bytes	Constant time per arithmetic operation	Approximate decimal representation; rounding errors may occur

For (n) bills, processing all bills takes (O(n)) time with either approach. Integer-paise calculations are generally preferable for billing because they represent whole paise exactly and apply a consistent rounding rule.

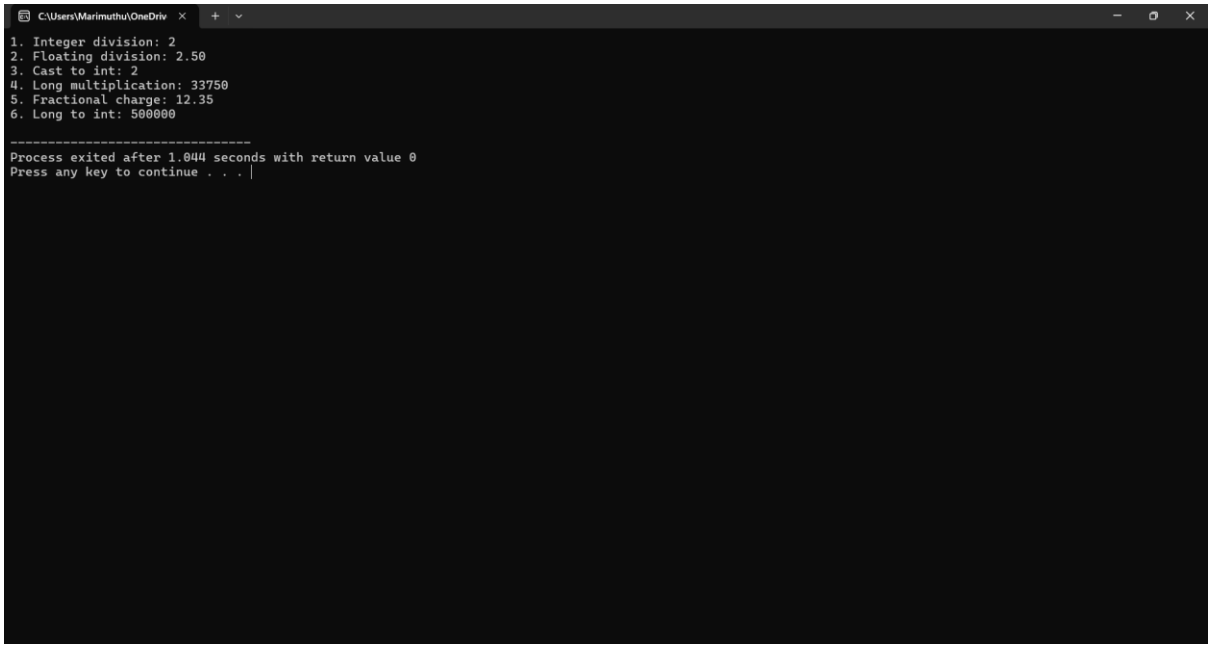
Code :

```
#include <stdio.h>

int main(void)
{
   int units = 150;
   long rate = 225;
   long charge = (long)units * rate;
   long amount = 500000L;
   double fractional = 12.345;
   printf("1. Integer division: %d\n", 5 / 2);
   printf("2. Floating division: %.2f\n", 5.0 / 2);
   printf("3. Cast to int: %d\n", (int)(5.0 / 2));
   printf("4. Long multiplication: %ld\n", charge);
```

```
    printf("5. Fractional charge: %.2f\n", fractional);
    printf("6. Long to int: %d\n", (int)amount);
    return 0;
}
```

Output :



Conclusion

Understanding implicit conversion, integer division, explicit casting and rounding is essential for accurate C calculations. Storing money as integer paise avoids many floating-point precision problems.